Query Processing
— Chapter 15

2025 年 12 月

School of Computer Science

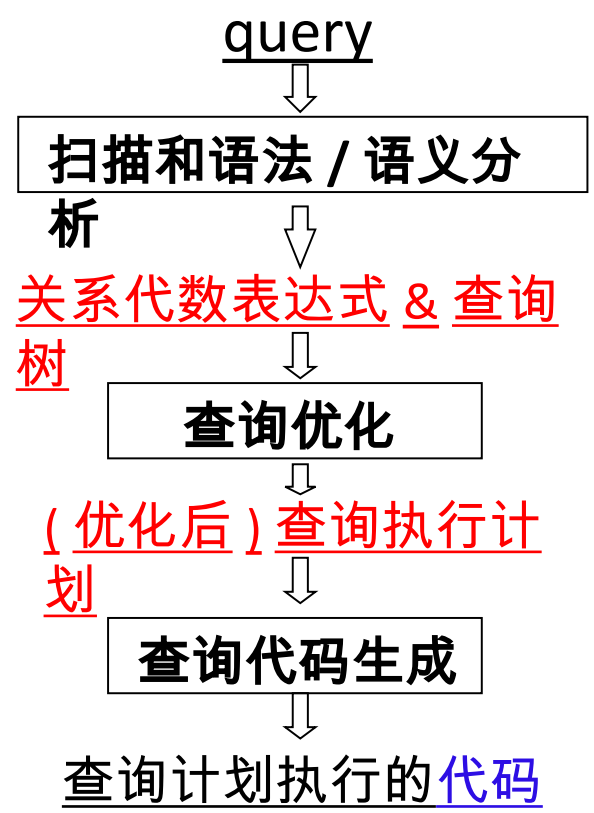
北京郵電大學

Query Processing and Optimization

- Query
  - operations on DB
  - requests for DB access
    - select, insert, delete, update, create, drop; read, write
- Chapter 15 Query Processing
  - how queries are processed,
- Chapter 16 Query Optimization
  - the lowest-cost evaluating method of a given query

Ch 15,16

Query processing / DBMS



Chapter 15 Query Processing

- **Def: Query processing** refers to activities (DBMS)

  extracting data from DB, including

  - **1. translation** of queries in DB languages **into** expressions that can be used at the physical level of the system
    - **SQL statement** into a initial **relational algebra** expression
  - **2. query-optimizing** transformations
  - **3. actual evaluation** of queries (執行)

15.1 Overview

- **Steps**
  in query processing

```
select   salary
from     instructor
where    salary <2500
```

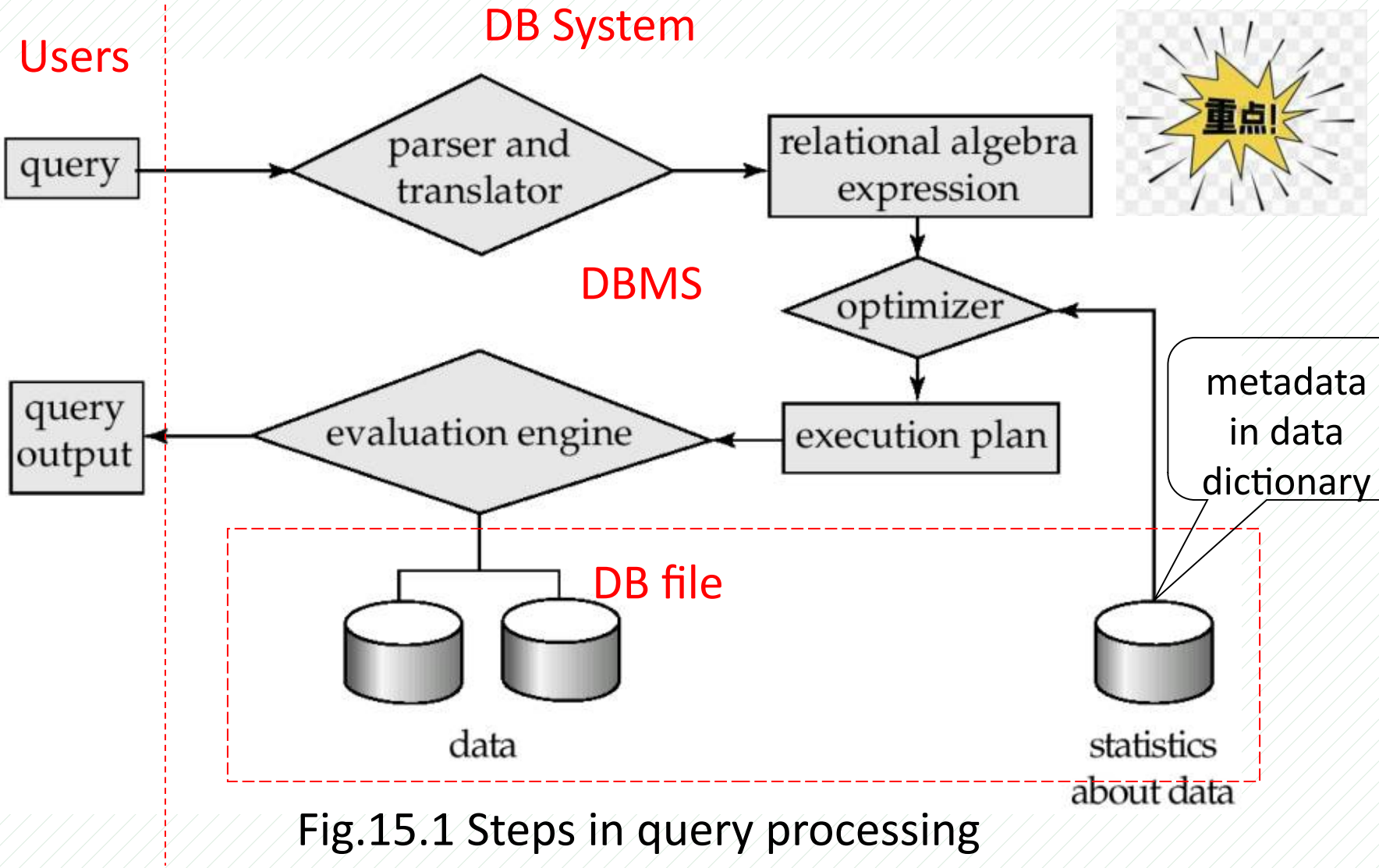


Fig.15.1 Steps in query processing

1.Parsing and translation

2.Optimization

3.Evaluation

Overview

- **Step1.Parsing and translation**
  - translate **query** into **parser-tree** 解析树
    - parser checks syntax, verifies the correctness of the relations
    - parser replaces the view with relations on which built
  - the **parser-tree** is then translated into **relational algebra**
- **Step2. Query Optimization**
  - choose the **lowest cost plan** among equivalent **query evaluation plans**,
  - the cost is **estimated** using statistical information in data dictionary
    - e.g. the number of tuples, size of tuples, etc

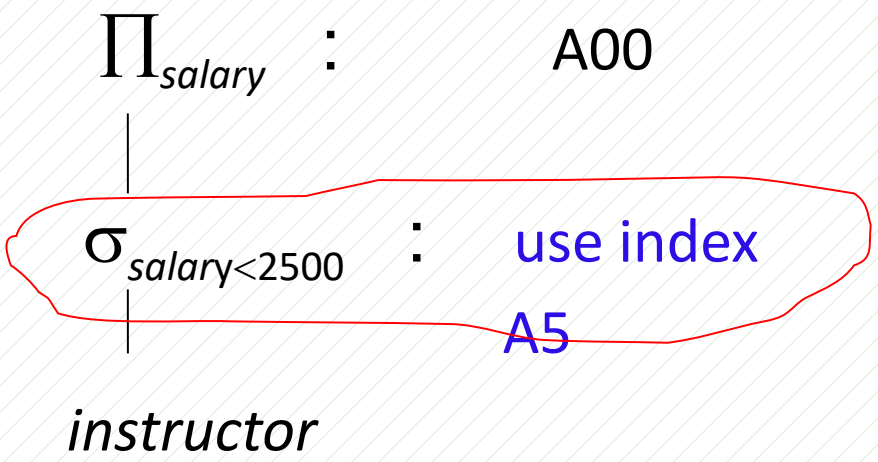
Optimization

- A relational algebra expression has equivalent expressions
  - $\sigma_{salary<75000}(\Pi_{salary}(instructor))$ is equivalent to $\Pi_{salary}(\sigma_{salary<75000}(instructor))$
- Each relational algebra operation can be evaluated with algorithms
  - e.g. the select operation can be evaluated using A1, A2, …, A11 algorithms
  - e.g. nested-loop join, merge join, hash join operation

Optimization

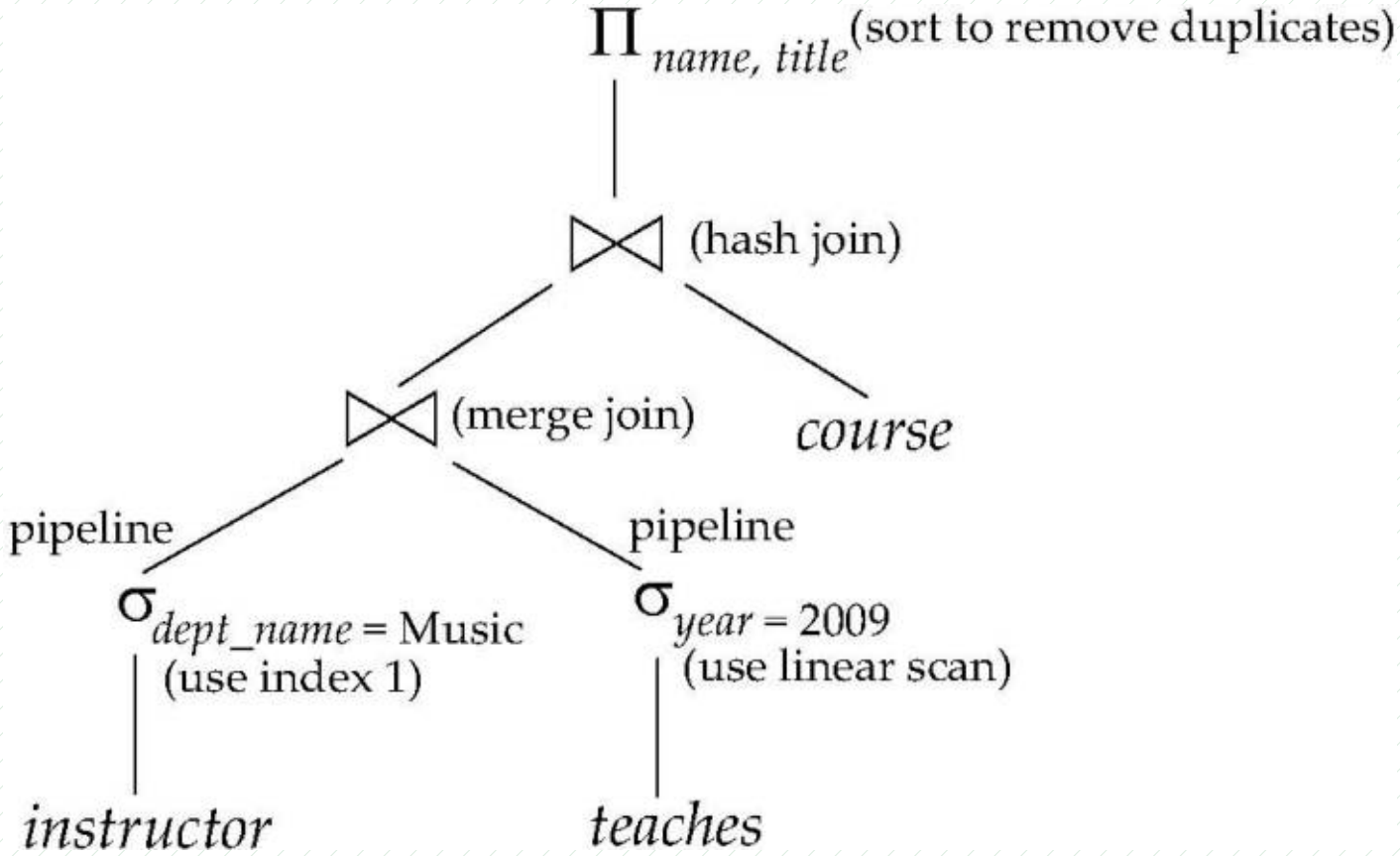
- **Def**: the **evaluation-plan is** an annotated expression specifying the detailed evaluation strategy, such as index, evaluation algorithms, etc.
  - use an index on *salary* to find ***instructors*** with ***salary*** < 2500,
  - perform the relation scan and discard instructors with ***salary*** ≥ 2500

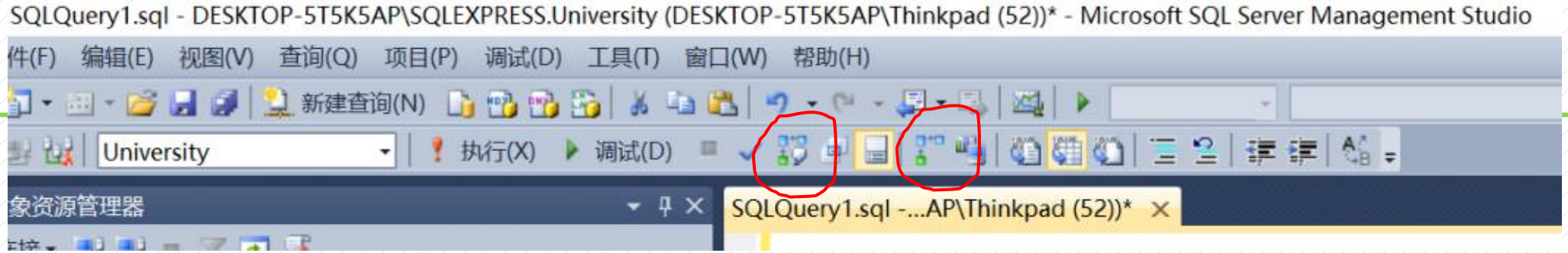
```
select  salary
from    instructor
where   salary <2500
```



Overview

- **Step3. Evaluation plan execution:** the query-execution engine
  - takes an optimized query-evaluation plan,
  - executes the optimized plan, and returns answers to the query

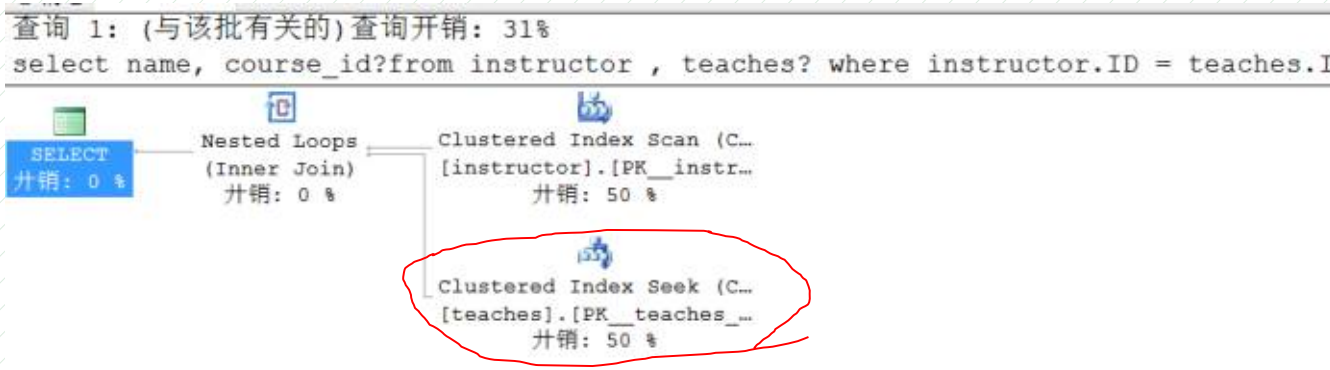


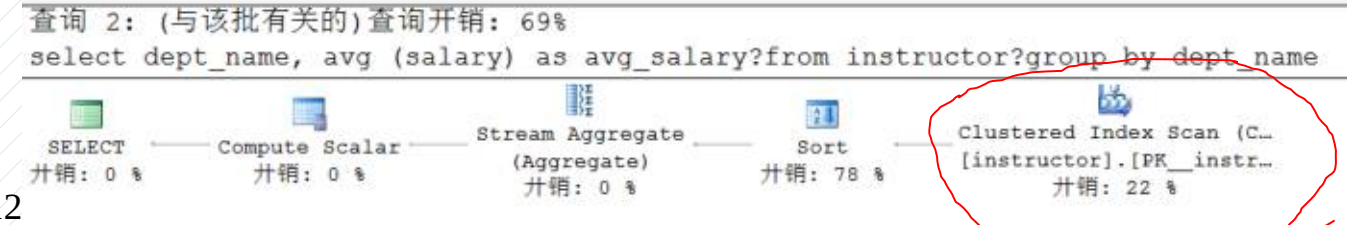



```sql
select name, course_id from instructor , teaches
where instructor.ID = teaches.ID
      and  instructor. dept_name = 'Art'

select dept_name, avg (salary) as avg_salary
from instructor group by dept_name;
```

在 SQL Server 上，观察 SQL 语句的查询执行计划和执行 cost 估计





15.2 Measures of Query Costs

- **Def: Cost** is measured as the total elapsed time for answering query
  - Factors contribute to time cost
    - disk access,
    - network communication
- **Def: Disk access** is the predominant cost, measured by
  - 1. number of **seeks** * average-seek-cost
    - 数据定位时间，需要访问的 records 在 DB file 中的位置
  - 2. number of block **read** * average-block-read-cost
  - 3. number of block **written** * average-block-write-cost
    - cost to write is greater than cost to read
    - data is read back after being written to be ensured successful

面向磁盘的数据库性能开销分析

- 假设一次业务处理（如 SQL 查询）总开销是 100% ，其中
  - 7% 不到，真正处理**业务逻辑**
  - 34% ，用于**缓冲区**管理如缓冲区的加载替换、地址转化等
  - 14% ，处理对于**内存数据**结构互斥访问加锁 Latching
  - 16% ，处理事务**并发访问**时，事务处理加锁 Locking
  - 12% ，处理日志 Logging
  - 16% ，用于对 **B 树索引**的处理

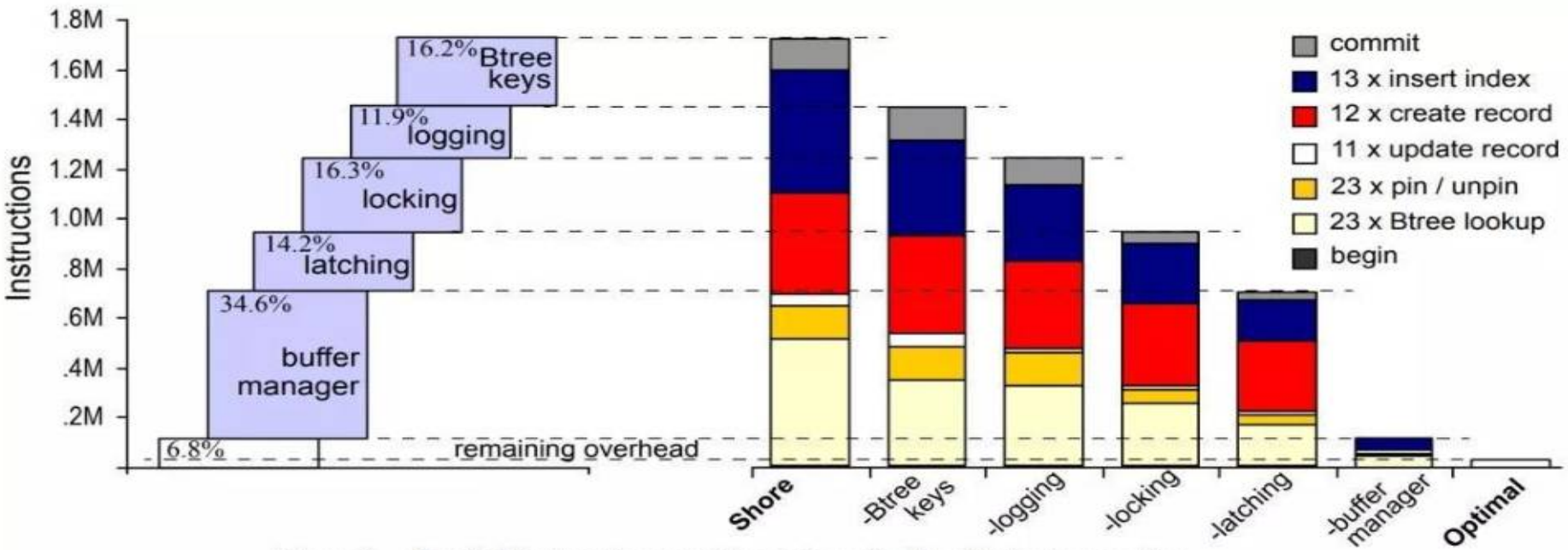
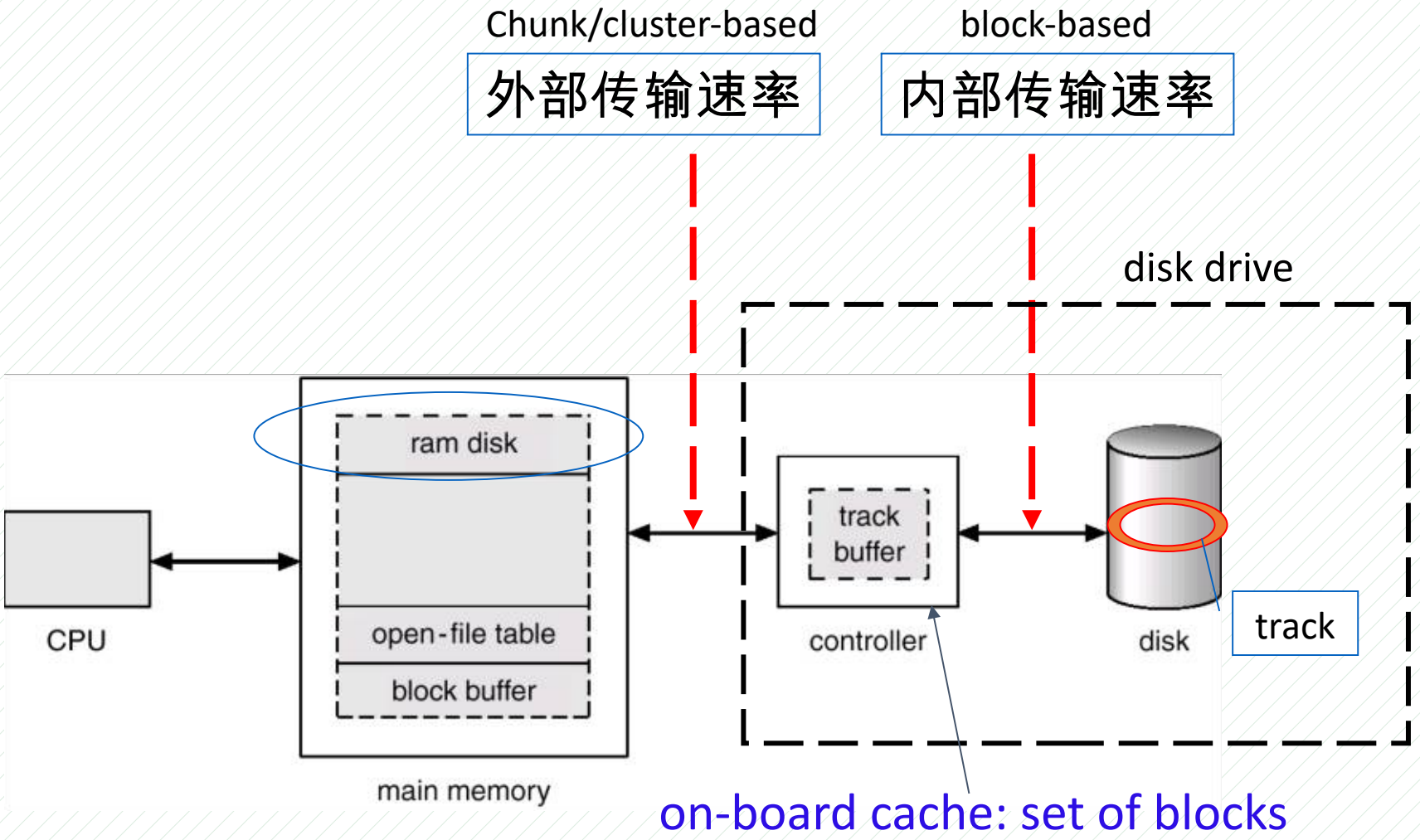


Figure 6. Detailed instruction count breakdown for New Order transaction.

Magnetic Hard Disk Mechanism



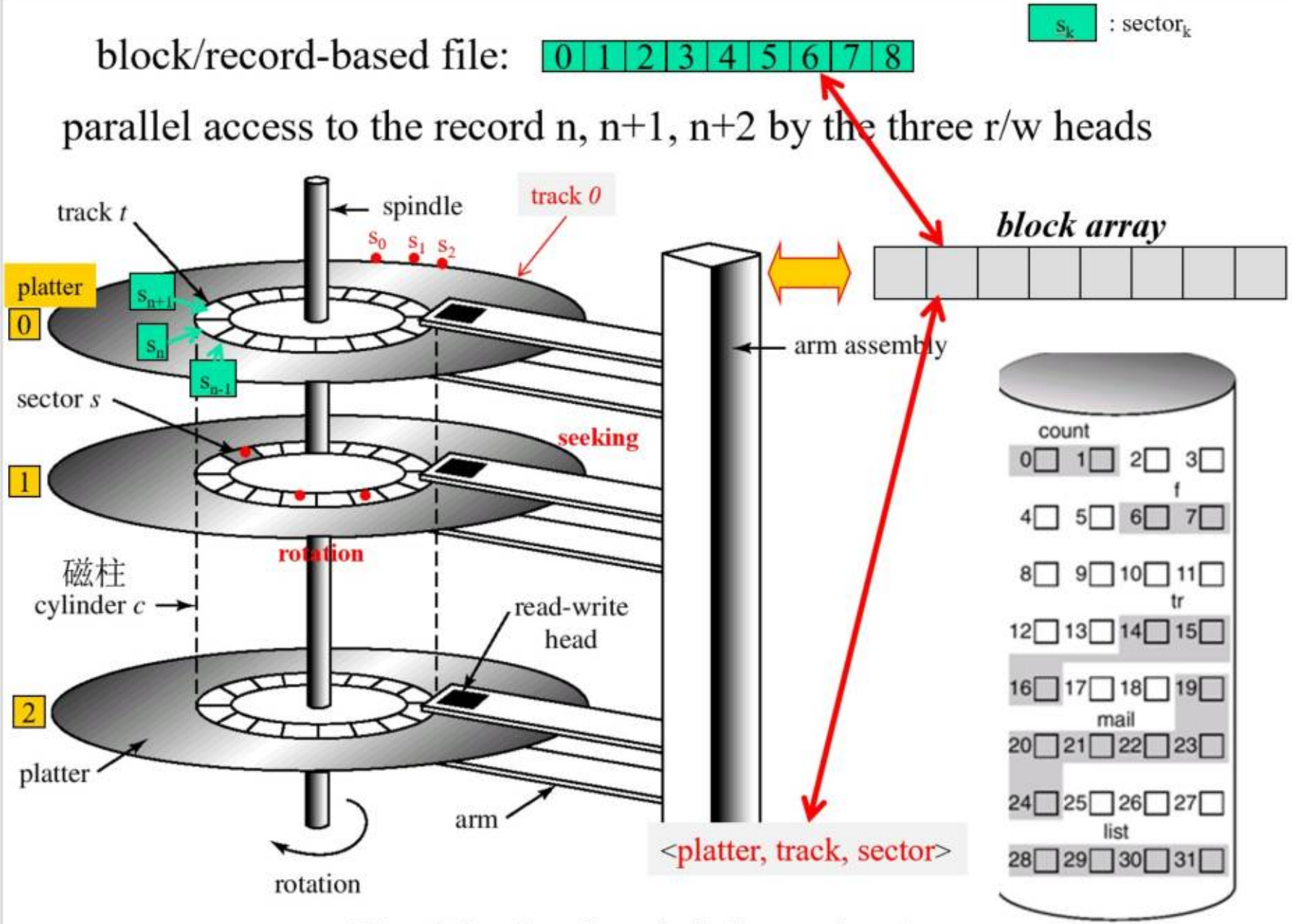


Fig. Moving-head disk mechanism

硬盘转速：3600 、 5400 或 7200 奇怪数字？

美国交流电 60Hz ，
1 分钟 =60 秒，
60Hz *1 转 /Hz * 60 秒 / 分钟 =3600 转 / 分钟
5400RPM=1.5*3600RPM
7200RPM=2*3600RPM

Measures of Query Costs

- **Cost measures**
  - **number of block transfered** from disk t_T
  - **number of seeks** t_S
    - t_T – time to transfer one block
      - disk 的内部传输速率
    - t_S – time for one seek
      - 包括： 寻道时间，旋转延迟
    - *time cost* for b block transfers plus s seeks

      $b * t_T + s * t_S$

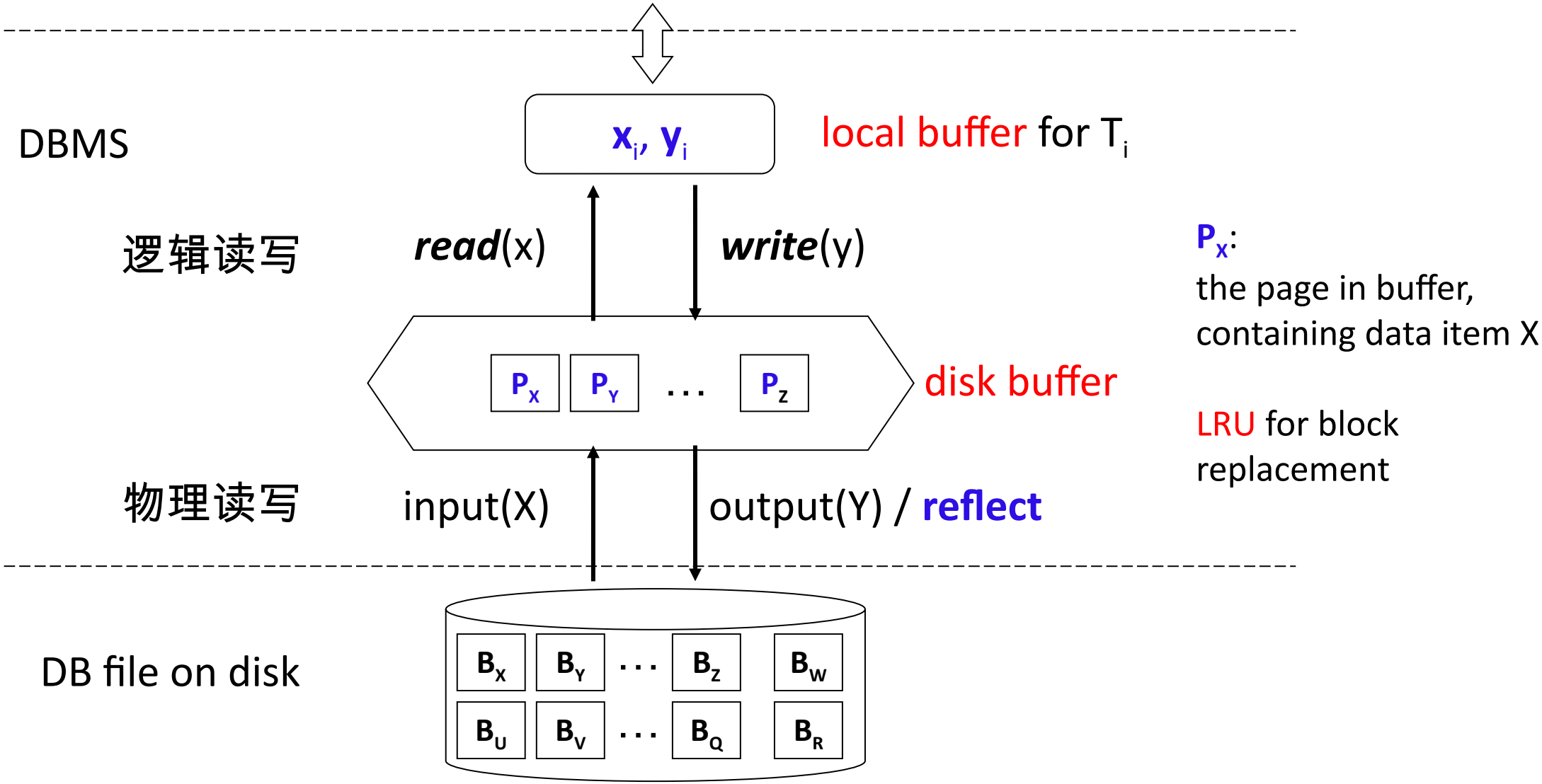
cost = < b, S >

Measures of Query Costs

- Algorithms can **reduce** disk **I/O** by using extra **buffer space**
  - amount of **real memory** available to buffer depends on concurrent queries and processes, known **during execution**
    - using the **worst case estimates**
    - assuming the **minimum amount of memory** needed for the operation
- Required data may be buffer resident already, avoiding disk I/O
  - hard to take into account for cost estimation

Transaction data accesses on data item **x** and **y** issued by T_i,

e.g. **select, insert, delete, update**, …



P_x:
the page in buffer, containing data item X

LRU for block replacement

15.3 Selection Operation

- Selection **operation σ** on relation ***r*** by **file scan**
  - **locating and scanning** the file in which ***r*** is stored
    to **retrieving** the *records* satisfying the selection conditions
- e.g. $\Pi_{salary}(\sigma_{salary>2500}(instructor))$

```
select  *
from    instructor
where   salary>2500
```

Selection Operation

- Types of query conditions (查询条件类型)
  - equality(等值), e.g. salary = 100
  - range (范围) , e.g. salary between 50 and 400
  - comparison (比较), - 也属于范围查询 ,e.g. salary >300
- **File scan algorithms** **cost estimation!**
  - **linear search**/scan – A1
  - selections using **indices** – A2, A3, A4
  - selections involving **comparisons** – A5, A6
  - **complex** selections – A7, A8, A9, A10
    - 多条件查询

A1: File scan by linear search

- Algorithm **A1** (linear search).
  - **scan each** block and **test** all records to see whether satisfy selection condition
  - cost = b_r block transfers + 1 seek

    = $b_r * t_T + t_S$

    - b_r: the number of blocks // 扫描全部 blocks ，找到全部满足查询条件数据
  - if selection is on the key attribute, it can stop on finding record equality,
  - average case:

    cost = (b_r/2) block transfers + 1 seek

    = (b_r/2) * t_T + t_S

A1: File scan by linear search

- **Linear search** can be applied regardless of
  - **selection** condition
  - **ordering** of records
  - **availability** of indices
- **suitable for**
  - seeking on a relational table organized as the **heap file**
    - scan on ***account_number*** or ***balance*** in *account*
  - seeking on a **non-index** attribute (non-search-key attribute) in a relational table organized as the **sequential file**, e.g. B/B+-tree file
    - index scan (索引扫描) on ***balance*** in *account*

A2: Selections Using Indices

- **Index scan/seek** – search algorithms with **an index**
  - selection condition on **search-key** of index
  - key-attributes or **non-key-attributes**

    i.e. ***instructor*** (***ID***, name, ***dept-name***, salary)
- **A2** (**clustering/primary index, B+-tree Index, equality on key**)

  Retrieve a single record, using B^+-tree as the clustering/primary index
  - e.g. seek on primary key ***ID*** =45565
  - cost = $(h_i + 1) * (t_T + t_S)$
    - h_i : height of the tree
    - t_T : time to transfer one block
    - t_S : *time for one seek*

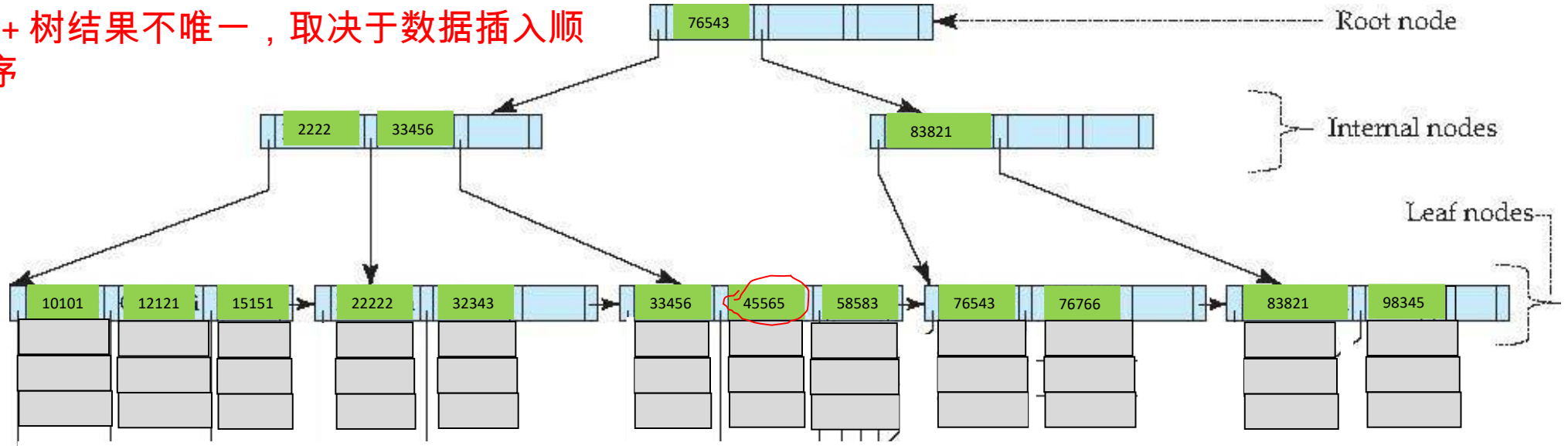
(Where h_i denotes the height of the index.) Index lookup traverses the height of the tree plus one I/O to fetch the record; each of these I/O operations requires a seek and a block transfer.

文件记录直接存储在叶节点

B+-Tree (*n*=4) Index Sequential File

Primary/Clustering index on **ID**

B+ 树结果不唯一，取决于数据插入顺序




```
create table instructor
      (ID integer, primary key;
       name       char(10);
       department  char(10);
       salary    integer;)
Or:  create clustered index ID
        Index on instructor(ID)
```

10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	80000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	60000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

A3: Selections Using Indices

- **A3** (**primary/clustering index, B+-tree Index**, equality on **non-key**)

  Retrieve **multiple** records that satisfy select conditions

  - e.g. seek on non-key attribute branch_name in *account*

    and branch_name= ‘ Perryridge ’ , **3** 个满足条件的记录

  - records are located on consecutive blocks
  - cost = $h_i * (t_T + t_S) + t_S + t_T * b$
    - b = number of blocks containing matching records

      e.g. for branch_name=‘Perryridge’, b=3

One seek for each level of the tree, one seek for the first block. Here b is the number of blocks containing records with the specified search key, all of which are read. These blocks are leaf blocks assumed to be stored sequentially (since it is a clustering index) and don’t require additional seeks.

A4: Selections Using Indices

- **A4** (secondary index, **B+-tree Index**, equality on **key**).
  - 1. if equality condition is on **key**, only a single record is retrieved
    - e.g. seek on *name*=Gold in ***instructor***
    - *cost* = $(h_i + 1) * (t_T + t_S)$,
  - 2. if equality condition is on **non-key,** multiple records (n records) are retrieved
    - e.g. seek on *salary*=8000 in ***instructor*** , n=3
    - each of *n* matching records may be resident on a different block, which may result in on I/O operation per retrieved record
    - cost = $(h_i + n) * (t_T + t_S)$

(Where n is the number of records fetched.) Here, cost of index traversal is the same as for A3, but each record may be on a different block, requiring a seek per record. Cost is potentially very high if n is large.

Secondary Index

非根、非叶结点的儿子结点个数k:
$2=\lceil n/2 \rceil \le k \le n=4$

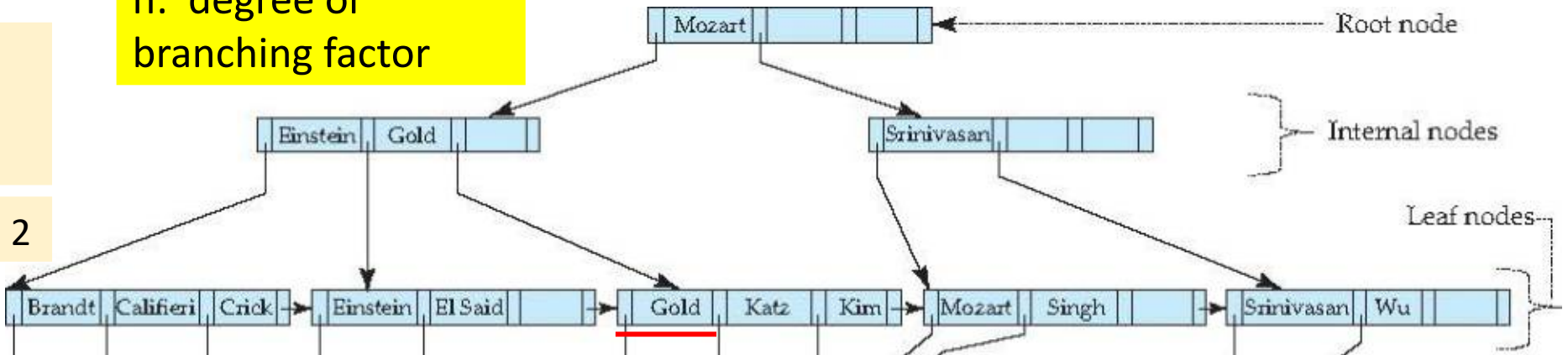
Example of B$^+$-Tree (n=4)

n: degree or branching factor

{Brandt, Califieri, Crick, Einstein, El Said, Glod}

block#: 5,3, 4,1,2, 2

```
select *
from instructor
where name <= 'Gold'
```



12 records

instructor(***ID***, ***name***, *dept_name, salary*)

10101	Srinivasan	Comp. Sci.	65000	block0
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	block1
22222	Einstein	Physics	95000	
32343	El Said	History	80000	block2
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	block3
58583	Califieri	History	60000	
76543	Singh	Finance	80000	block4
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	block5
98345	Kim	Elec. Eng.	80000	

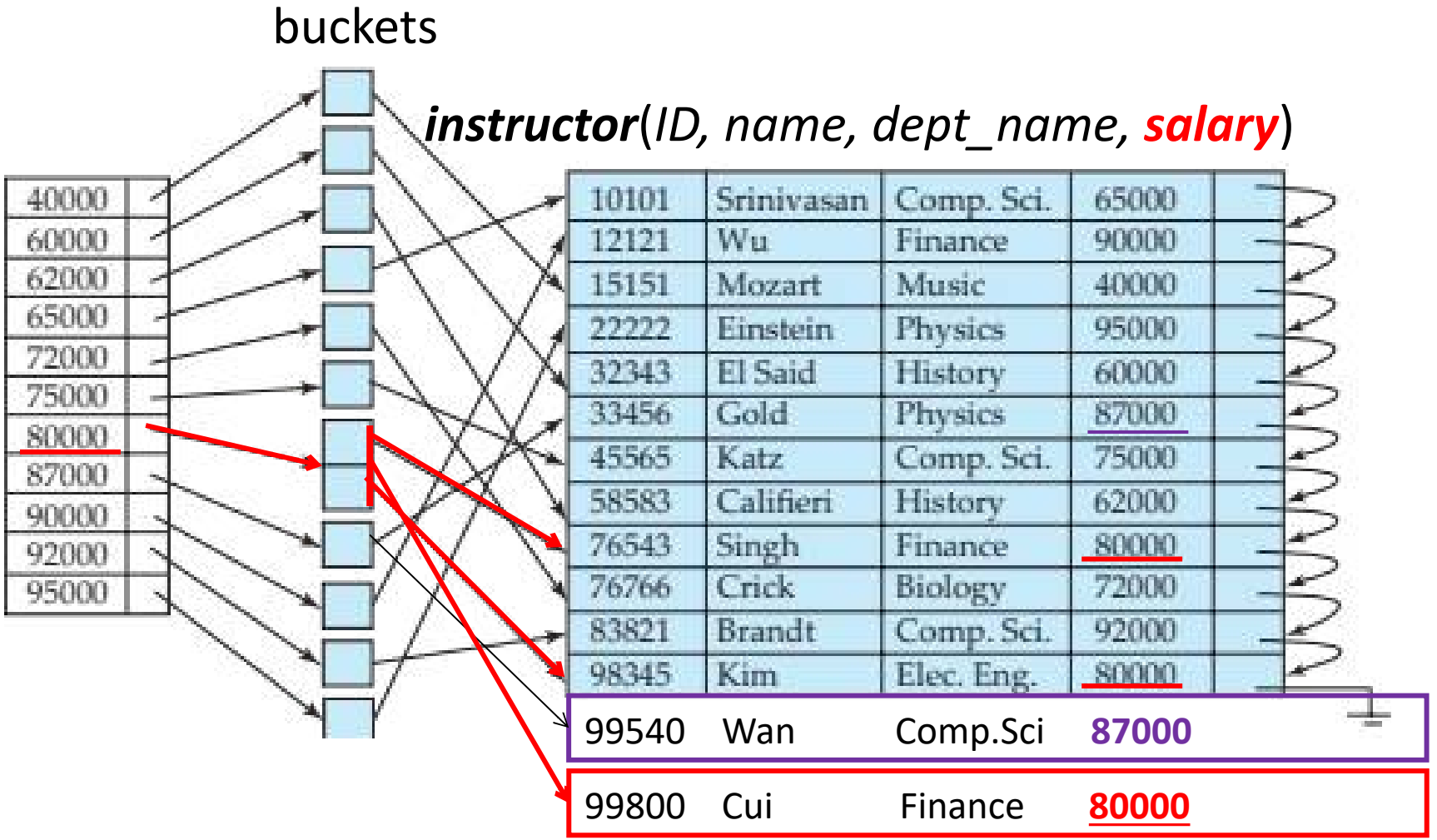
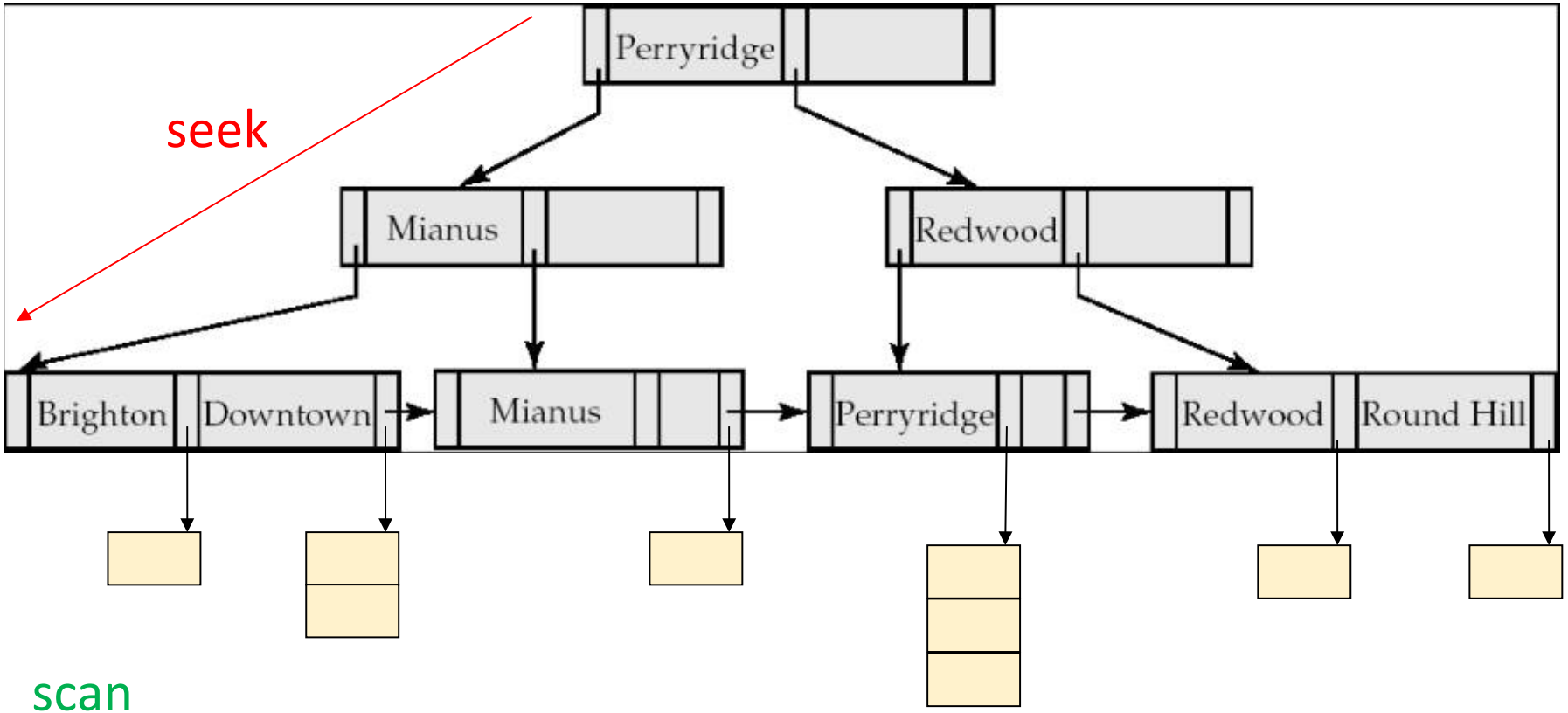


Fig. 14.6 Secondary index on *salary* field of *instructor*

聚集索引树：索引 + **数据**　访问方式：

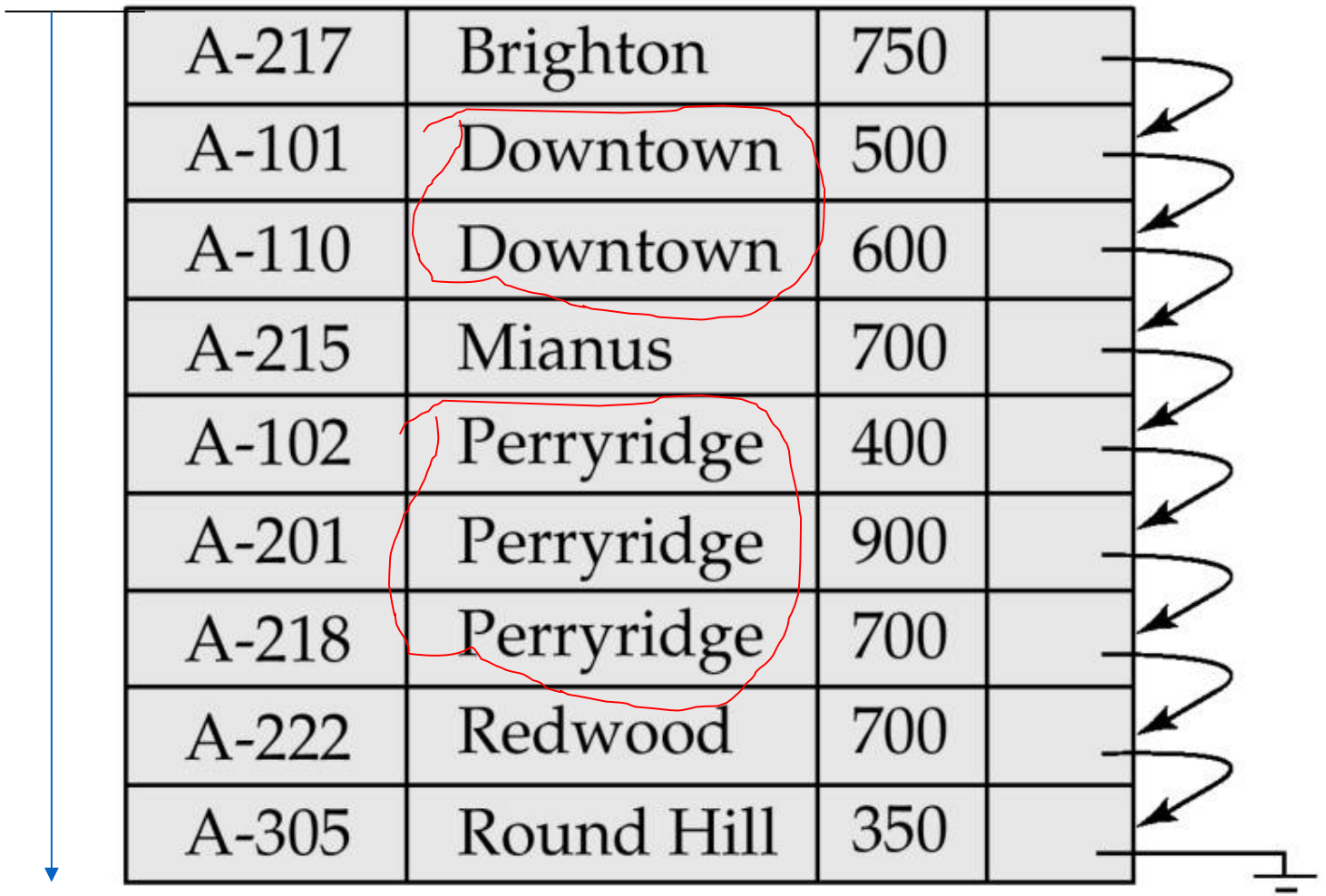
1. 聚集索引查找 seek on index-attribute ， e.g. *branch-name*, n 分查找 seek
2. 聚集索引扫描 scan on non-index-attribute, e.g. *balance* ，线性扫描 sacn

account (account-number, branch-name, balance)



未建立索引，堆文件
访问方式：线性扫描、表扫描

***Account*(*account_number*, *branch-name*, *balance*)**



A-217	Brighton	750	
A-101	Downtown	500	
A-110	Downtown	600	
A-215	Mianus	700	
A-102	Perryridge	400	
A-201	Perryridge	900	
A-218	Perryridge	700	
A-222	Redwood	700	
A-305	Round Hill	350	

A5: Selections Involving Comparisons- Range Search

- **Implement** selections: the form $\sigma_{A \le V}(r)$ or $\sigma_{A \ge V}(r)$ using
  - a **linear** file scan,
  - **indices**
- **A5** (**clustering/primary index, B+tree Index, comparison**). (Sorted on A)

  关系中的各个记录 / 元组已经按照属性 A 排序，存储在文件中
  - for $\sigma_{A \ge V}(r)$ use index to find the first tuple $\ge v$ and scan relation sequentially
  - for $\sigma_{A \le V}(r)$ just scan relation sequentially till the first tuple > *v;* do not use index
  - cost = $h_i * (t_T + t_S) + t_S + t_T * b$
    - *b*: number of blocks containing matching records

A6: Selections Involving Comparisons

- **A6** (**secondary index, B+tree Index, comparison**).
  - e.g. $\sigma_{\text{name} \geq \text{Gold}}$ (*instructor)* in
  - for $\sigma_{A \geq V}(r)$ use **index** to find first **index entry** $\geq v$ and scan index sequentially, to **find pointers** to records
  - for $\sigma_{A \leq V}(r)$ scan **leaf pages** of index finding pointers to records, till first entry > v
  - cost = $(h_i + \mathbf{n}) * (t_T + t_S)$
  - in either case, retrieve records that are pointed
    - requires **an** I/O for **each** record
    - **linear file scan** may be cheaper

	Algorithm	Cost	Reason
A1	Linear Search	$t_S + b_r * t_T$	One initial seek plus b_r block transfers, where b_r denotes the number of blocks in the file.
A1	Linear Search, Equality on Key	Average case $t_S + (b_r/2) * t_T$	Since at most one record satisfies the condition, scan can be terminated as soon as the required record is found. In the worst case, b_r block transfers are still required.
A2	Clustering B$^+$-tree Index, Equality on Key	$(h_i + 1) * (t_T + t_S)$	(Where h_i denotes the height of the index.) Index lookup traverses the height of the tree plus one I/O to fetch the record; each of these I/O operations requires a seek and a block transfer.
A3	Clustering B$^+$-tree Index, Equality on Non-key	$h_i * (t_T + t_S) + t_S + b * t_T$	One seek for each level of the tree, one seek for the first block. Here b is the number of blocks containing records with the specified search key, all of which are read. These blocks are leaf blocks assumed to be stored sequentially (since it is a clustering index) and don't require additional seeks.
A4	Secondary B$^+$-tree Index, Equality on Key	$(h_i + 1) * (t_T + t_S)$	This case is similar to clustering index.
A4	Secondary B$^+$-tree Index, Equality on Non-key	$(h_i + n) * (t_T + t_S)$	(Where n is the number of records fetched.) Here, cost of index traversal is the same as for A3, but each record may be on a different block, requiring a seek per record. Cost is potentially very high if n is large.
A5	Clustering B$^+$-tree Index, Comparison	$h_i * (t_T + t_S) + t_S + b * t_T$	Identical to the case of A3, equality on non-key.
A6	Secondary B$^+$-tree Index, Comparison	$(h_i + n) * (t_T + t_S)$	Identical to the case of A4, equality on non-key.

Implementation of Complex Selections- Conjunction

- **Conjunction:** $\sigma_{\theta 1 \wedge \theta 2 \wedge \ldots \theta n}(r)$
- **A7** (**conjunctive selection using one index**)

  select *

  from instructor

  where *ID*=1000 and *dept_name*='Comp.Sci'

  - 1. select a combination of θ_i and algorithms A1 through A6 that results in the least cost for $\sigma_{\theta i}$ (r)
  - 2. test other conditions on tuple after fetching it into memory buffer
  - e.g. 1. **seek** the tuple with *ID*=1000 using index on ID

    2. **test** whether or not the *dept_name* of this tuple is 'Comp.Sci'

Implementation of Complex Selections- Conjunction

- **A8** (**conjunctive selection using composite index**)
  - use appropriate composite (multiple-key) index if available
  - e.g. using multiple-key index (*ID*, *dept_name*)

A9: Complex Selections- Conjunction

- **A9** (**conjunctive selection by intersection of identifiers**)
  - requires **indices** with record **pointers**
  - use corresponding index for **each condition**,
    take intersection of all the obtained sets of record pointers.
  - if some conditions do not have appropriate indices, apply test in memory

step1. 利用定义在每个查询条件θ_i的查询属性上的索引，查找满足θ_i的元组集合

step2. 取各个元组集合的交集

```
select   *
from     instructor
where    ID=1000   and   dept_name='Comp.Sci'
```

S1 ∩ S2

A10: Algorithms for Complex Selections

- **Disjunction:** $\sigma_{\theta 1 \vee \theta 2 \vee \ldots \theta n}(r)$.
- **A10** (**disjunctive selection by union of identifiers**).
  - applicable if all conditions have available indices
    - otherwise use linear scan
  - **use** corresponding index for each condition,
  - **take** union of all the obtained sets of record pointers.
  - **fetch** records from file

```
select   *
from     instructor
where    ID=1000    OR    dept_name='Comp.Sci'
```

S1 ∪ S2

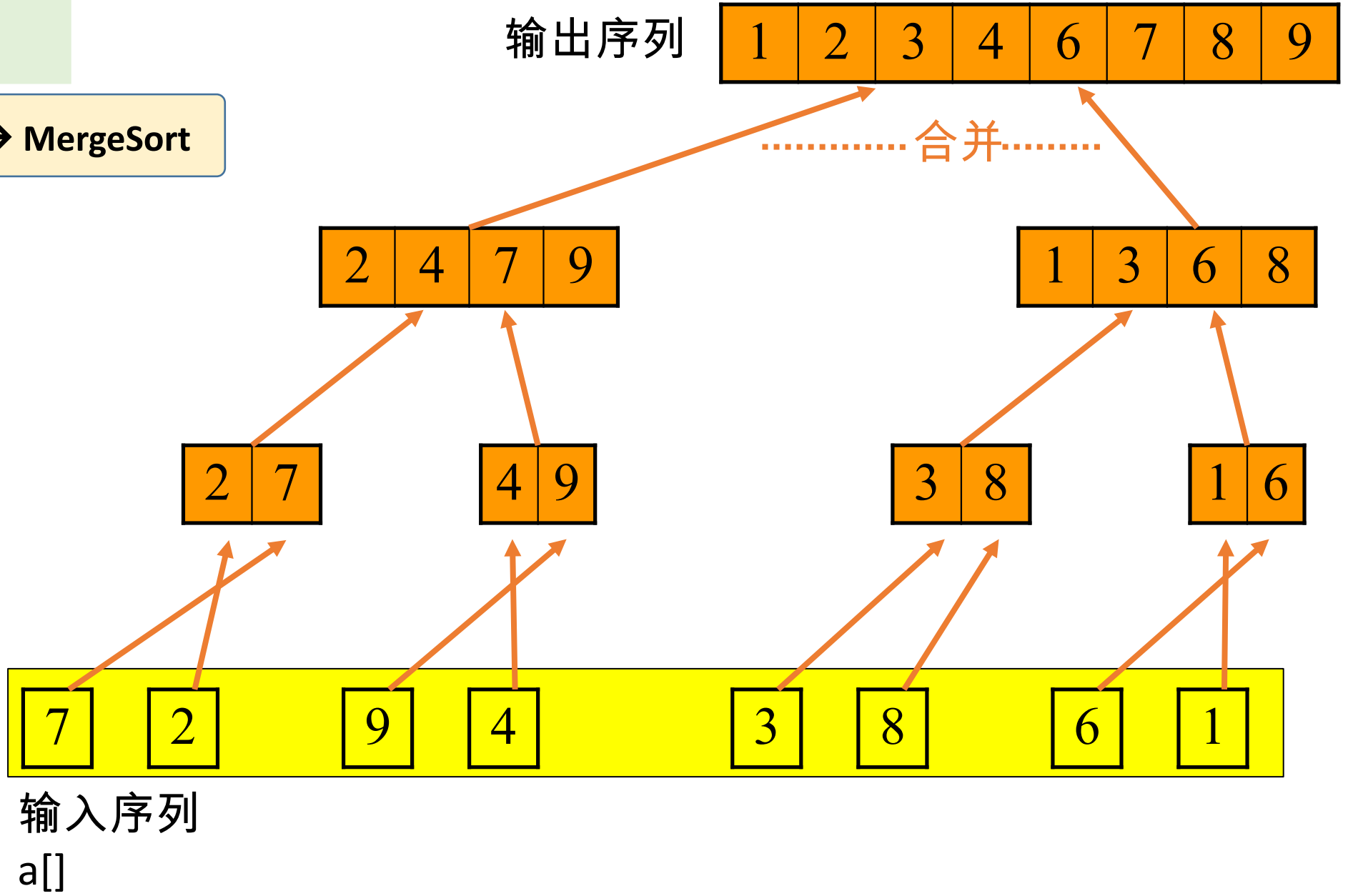
- **Negation:** $\sigma_{\neg\theta}(r)$
  - use **linear scan on file**
  - if very few records satisfy $\neg\theta$, and an index is applicable to θ
    - find satisfying records using index and fetch from file

15.4 Sorting

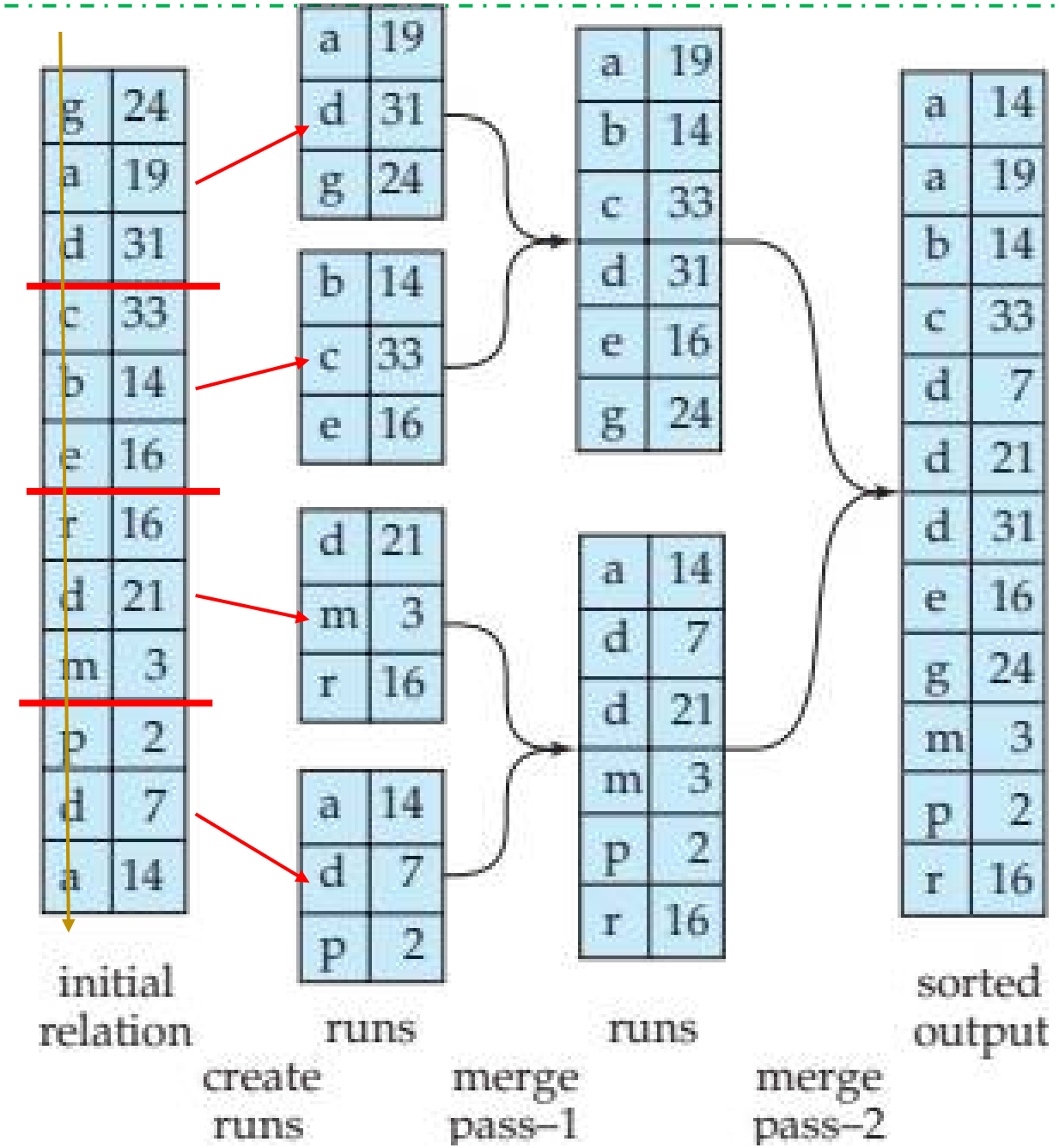
- **Sorting data** plays an important role
  - select **unique**(attri_A)-from-where-**order by**
    - sorting on query result
  - *instructor* **join** *department*
    - if two relations are first sorted, join operation can be implemented efficiently
- **Sorting DB**, records are ordered physically, for efficiently processing
  - primary-key **ID** in table **instructor** results a primary index/B+-tree index, in which the tuples are stored in ascending **order** of **ID**

非递归合并排序
—算法设计与分析

merge → MergePass → MergeSort



- Let M denote memory size
  - 1. **Create sorted runs.**
    Let *i* be 0 initially.
    Repeatedly do the following till the end of the relation:
    (a) Read *M* blocks of relation into memory
    (b) Sort the in-memory blocks
    (c) Write sorted data to run R_i;
    increment *i*.
    Let the final value of *i* be *N*
  - 2. Merge the runs



15.7 Evaluation of Expressions

- Evaluate an **expression** containing multiple evaluation primitives
  - key: how to process intermediate computing results
  - e.g. $\prod_{sarlay}(\sigma_{salary<2500}(instructor))$
- **materialization**(实体化 / 物化) and **pipeline** are used for expression evaluation
  - **Materialization**: generate results of an expression whose **inputs are relations** or are already computed, **materialize** (store) it on disk. Repeat.
  - **Pipelining**: pass on tuples to parent operations even as an operation is being executed

Materialization

- Principles 原理
  - start from the lowest-level, i.e. at the bottom of the tree, evaluate one operation at a time
  - the results of each evaluation (i.e. intermediate computing results) are stored in **temporal relations** on the disk for subsequent evaluation
    - serial evaluating
- e.g., compute and store in a **temporal relation** at first

$$\sigma_{salary<2500}(instructor)$$

- Cost of writing results to disk and reading them back can be quite high
- overall cost =
  sum of costs of individual operations + cost of writing intermediate results to disk

Materialization

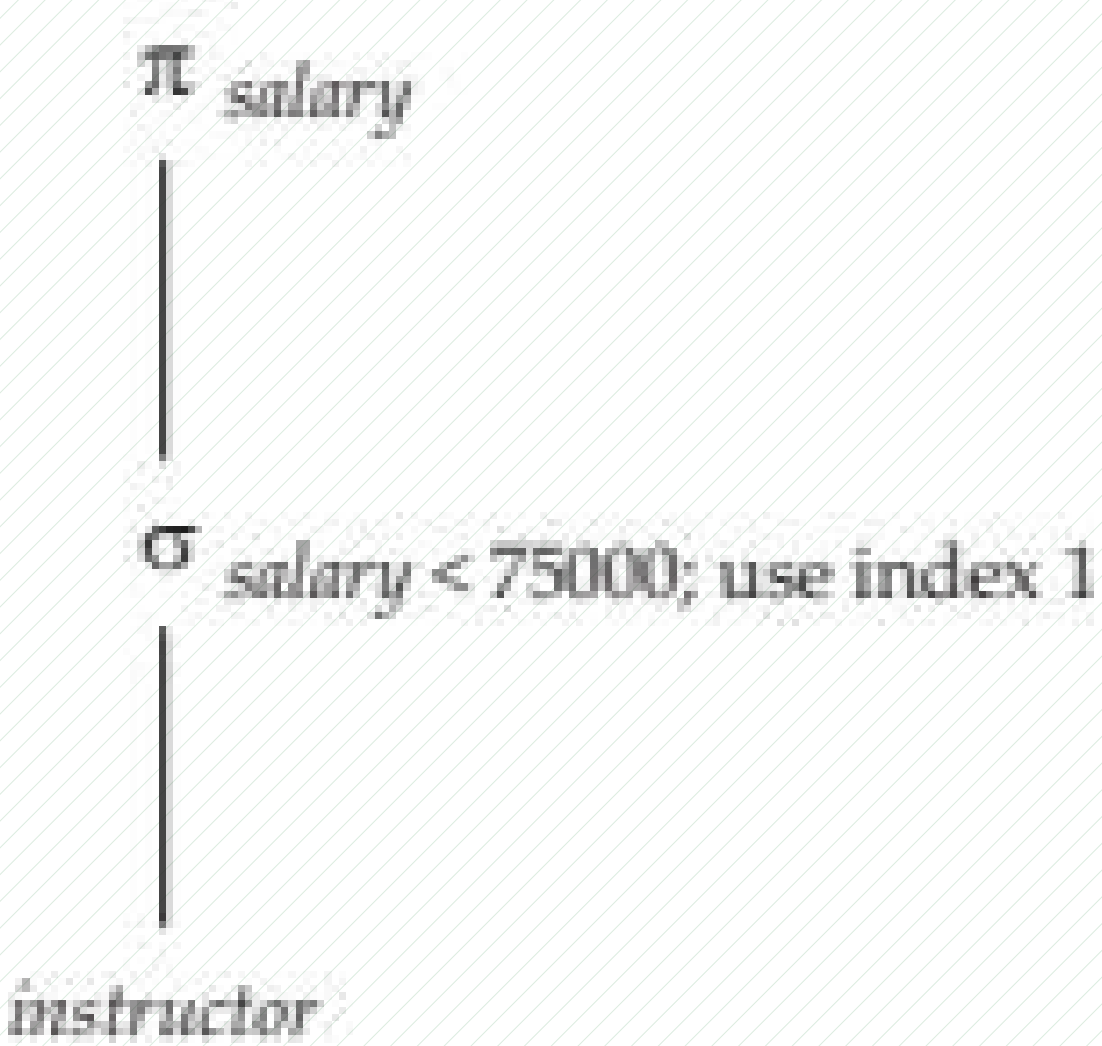


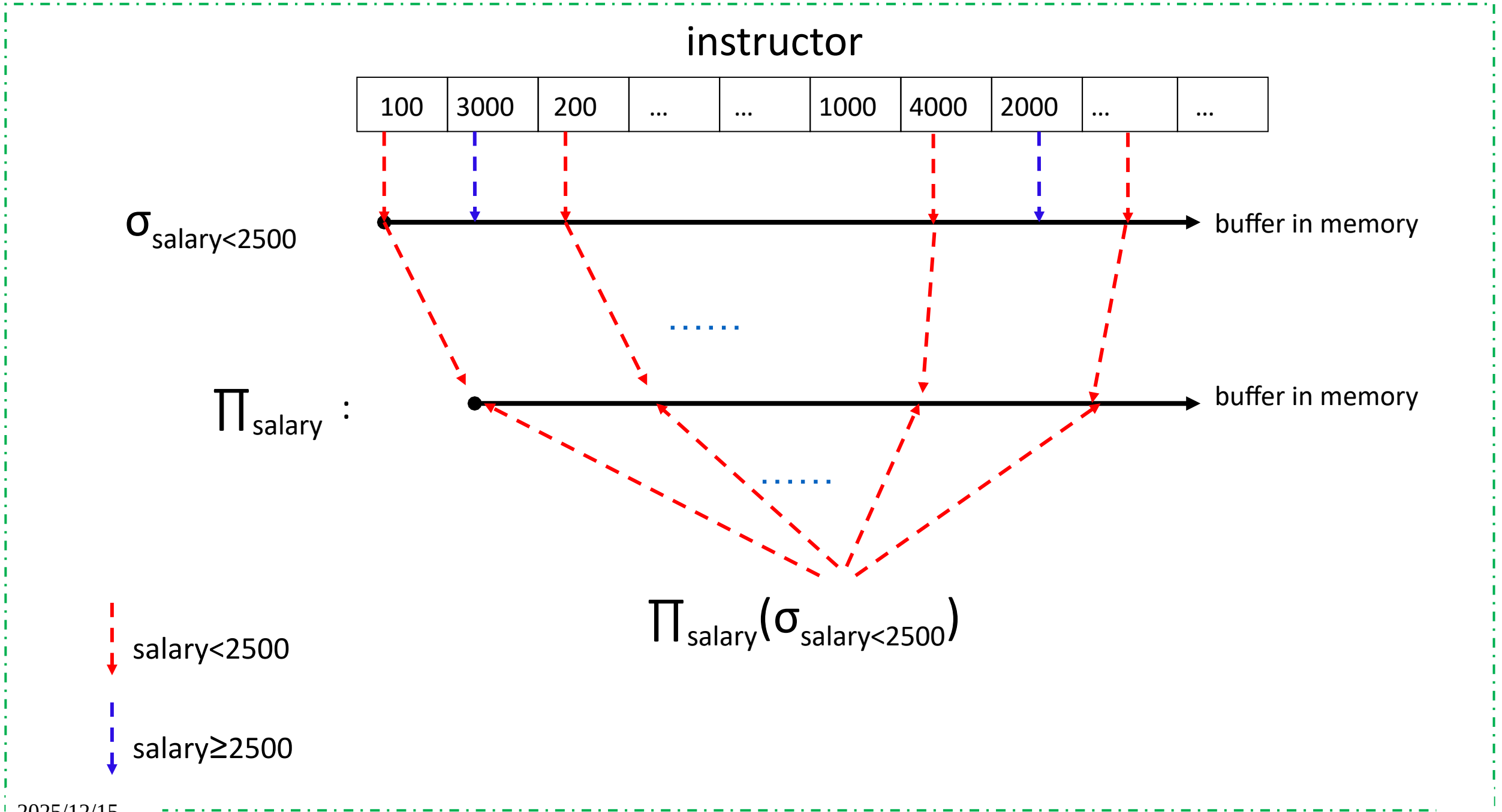
Fig. 15.0.2 Pictorial representation of an expression

Pipelined

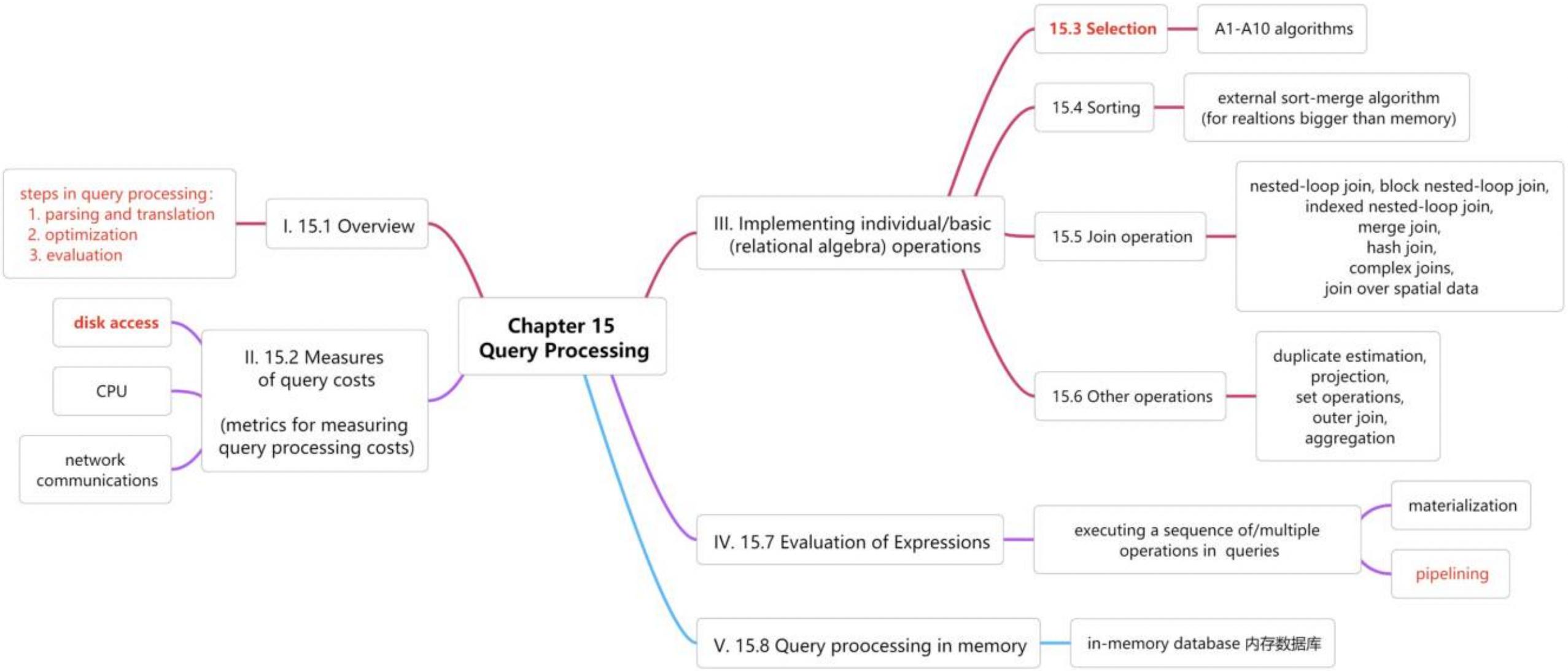
- Principles of pipelined evaluation
  - evaluate several operations simultaneously in a pipeline, with the results of one operation passed to the next, without need to store temporary relations in disk
    - parallel evaluating
- Much cheaper than materialization:
    - no need to store a temporary relation to disk
- e.g., in the expression tree in Fig.15.0.2, don't store result of

$$\sigma_{salary<2500}(instructor)$$

  - instead, pass tuples directly to the join; similarly, don't store result of join, pass tuples directly to projection.


instructor
100
3000
200
...
...
1000
4000
2000
...
...
σsalary<2500
buffer in memory
Πsalary :
buffer in memory
Πsalary(σsalary<2500)
salary<2500
salary≥2500

Outline


Chapter 15
Query Processing
I. 15.1 Overview
steps in query processing:
1. parsing and translation
2. optimization
3. evaluation
II. 15.2 Measures of query costs
(metrics for measuring query processing costs)
disk access
CPU
network communications
III. Implementing individual/basic (relational algebra) operations
15.3 Selection
A1-A10 algorithms
15.4 Sorting
external sort-merge algorithm (for realtions bigger than memory)
15.5 Join operation
nested-loop join, block nested-loop join, indexed nested-loop join, merge join, hash join, complex joins, join over spatial data
15.6 Other operations
duplicate estimation, projection, set operations, outer join, aggregation
IV. 15.7 Evaluation of Expressions
executing a sequence of/multiple operations in queries
materialization
pipelining
V. 15.8 Query proocessing in memory
in-memory database 内存数据库

Thanks for your attention